KSUG.AI #49 · SINGAPORE · 21 OCTOBER 2026

jina-ocr-v1

OCR

Faster Document Parsing on Low-Budget GPUs

OCR used to mean recognising characters. Today it usually means a model that reads the whole page and writes it out as Markdown.

Sa Zhang · Product marketing, inference at Elastic (Jina)

For 60 years, OCR meant recognising characters

1968

OCR-A

A typeface drawn so machines could read it

1976

Kurzweil Reading Machine

Reads printed pages aloud for blind readers

1985 · 2005

Tesseract

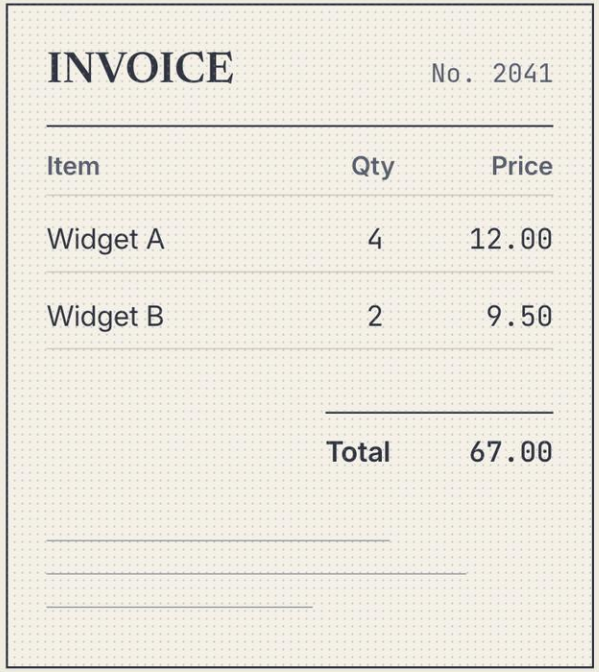
Built at HP; open-sourced in 2005

Characters got solved. Tables, columns and reading order didn't.

Public record: OCR-A typeface (1968); Kurzweil Reading Machine (1976); Tesseract at HP (1985), open source (2005)

Classic OCR gives you the characters, not the structure

THE PAGE



CLASSICAL OCR OUTPUT

```
INVOICE   No 2041
Item Qty Price Widget
 A 4 12.00  Wid get
B 2 9.50 Tota l
67 .00  ~ ._ ' ,
```

WHAT RETRIEVAL NEEDS

```
# Invoice 2041

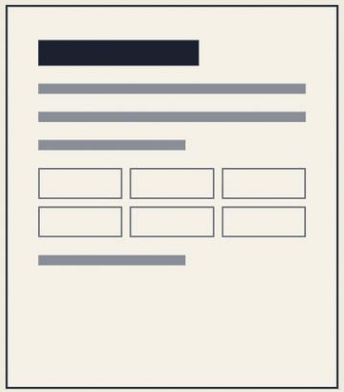
<table>
 <tr><th>Item</th><th>Qty</th>
     <th>Price</th></tr>
 <tr><td>Widget A</td><td>4</td>
     <td>12.00</td></tr>
 <tr><td>Widget B</td><td>2</td>
     <td>9.50</td></tr>
</table>

**Total:** 67.00
```

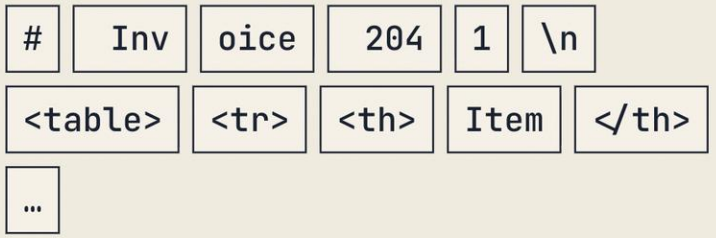
That's why so many scanned PDFs never make it into a search index.

Now a model reads the page image and writes out the text

PAGE IMAGE



WRITES THE TEXT, ONE TOKEN AT A TIME



MARKDOWN

```
# Invoice 2041

<table>
 <tr><th>Item…
</table>

$$ \sum … $$
```

Tables, formulas and reading order come out in one step.

Try it: one URL prefix, plus one header for scans

every request also sends `-H "Authorization: Bearer $JINA_API_KEY"`

```
curl https://r.jina.ai/<web-page>
```

WEB PAGE
Clean Markdown

```
curl https://r.jina.ai/<scanned-invoice.pdf>
```

SCAN, NO HEADER
No usable text

```
curl https://r.jina.ai/<scanned-invoice.pdf> \
  -H "X-Respond-With: jina-ocr-v1"
```

SAME SCAN + HEADER
Markdown, table intact

```
curl https://r.jina.ai/<two-column-paper.pdf> \
  -H "X-Respond-With: jina-ocr-v1" -H "X-Page: 3"
```

TWO COLUMNS + EQUATIONS
Right order, formulas as LaTeX

Try it: one URL prefix, plus one header for scans

TRY IT YOURSELF

Demo page

`debug000.jina.ai/ocr-harness`

Jina AI

`jina.ai`

What changed when OCR moved to language models

	UNTIL 2020	SINCE 2025
HOW IT'S MEASURED	~~Character error rate~~	Do the tables and layout come out right? (OmniDocBench, TEDS)
WHAT COSTS THE MOST	Finding text boxes, then reading them	Generating the output, token by token
WHAT IT'S BUILT ON	~~Computer-vision pipelines~~	Language models: olmOCR, DeepSeek-OCR, PaddleOCR-VL

The name stayed the same. The technology behind it is different.

Two ways to turn documents into text

A 2026 COURSE FROM DEEPLEARNING.AI AND LANDINGAI

Document AI: From OCR to Agentic Doc Extraction

Step by step

Layout model
trained on DocLayNet → OCR → Table model

e.g. Docling

You can check each step on its own.

One model, one pass

Page image → Model writes Markdown

`olmOCR` `DeepSeek-OCR` `Granite-Docling-258M`

`jina-ocr-v1`

Better on messy pages. Output is ready to index.

We build the second kind. Even Docling's team now has one: Granite-Docling.

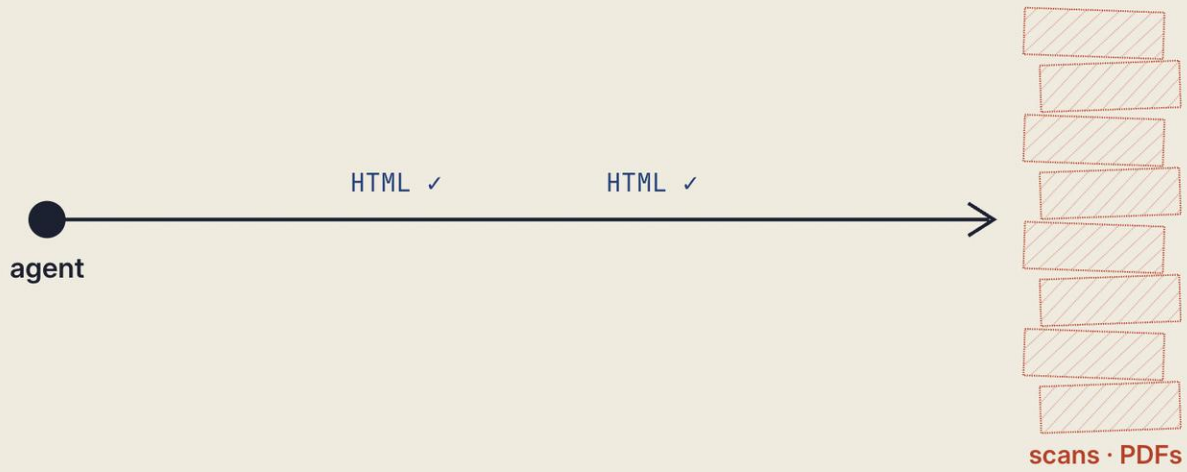
Agents need to read PDFs and scans, not just web pages

AI ENGINEER WORLD'S FAIR 2026, AS ATTENDEES REPORTED IT

01 RAG became infrastructure

02 Agents took centre stage

03 Vision & OCR got its own track



An agent that only reads HTML gets stuck on the first scanned PDF.

Sources: ai.engineer/worldsfair/2026 and published attendee recaps

Reader: add r.jina.ai before a URL, get Markdown

```
https://r.jina.ai/https://any-page.example
```

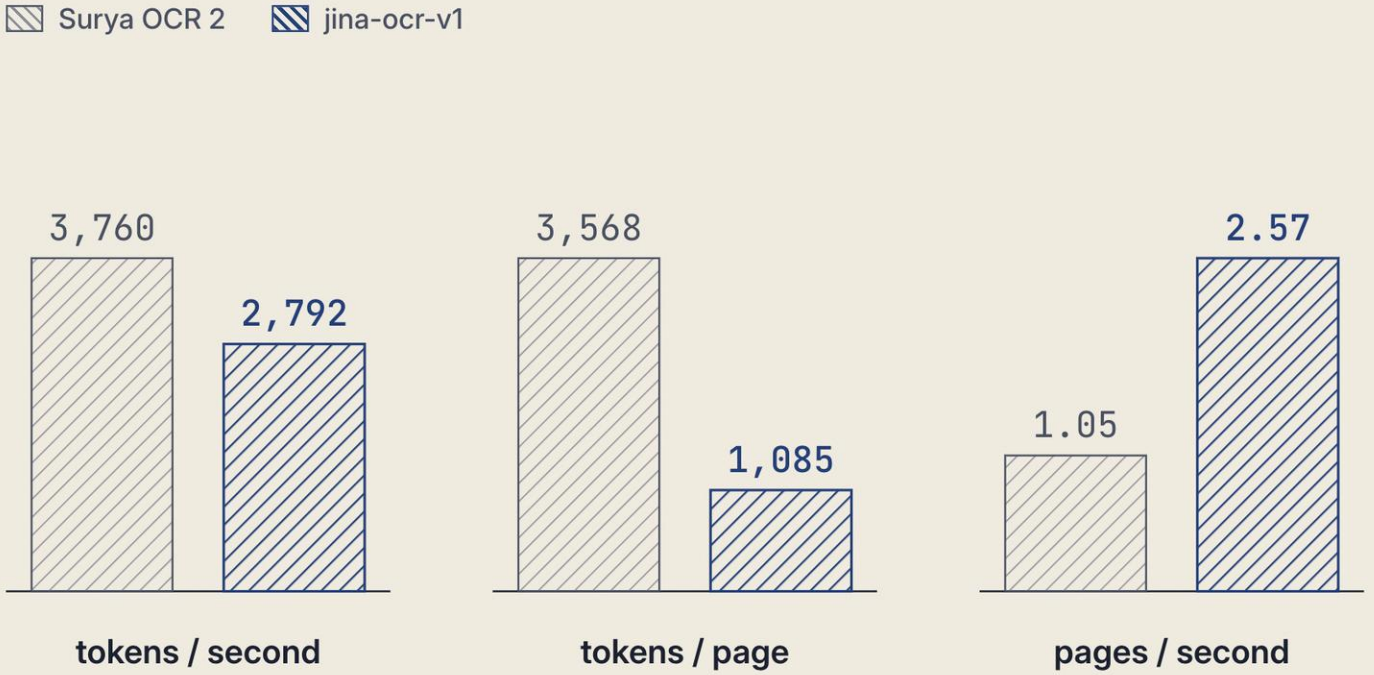
Web page → Markdown. Many developers already use this inside their agents.

```
https://r.jina.ai/https://scan.example/page.pdf
   -H "X-Respond-With: jina-ocr-v1"
```

Scanned PDF → Markdown. Same API key, same prefix, one extra header.

For scans, it's the same request with one more header.

Shorter output per page means more pages per second




```
pages/s
= tok/s ÷ tok/page
```

Surya generates tokens faster, but writes over 3× as many per page.

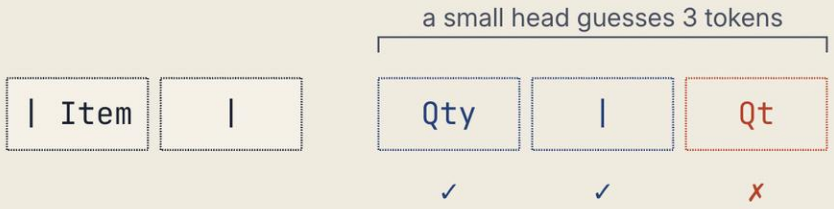
WHAT TO MEASURE

Measure how fast a page turns into Markdown you can index.

arXiv:2609.03181 · same-run throughput comparison · jina-ocr-v1 writes the shortest pages among systems above 83 on olmOCR-Bench

Faster and better, without a bigger model

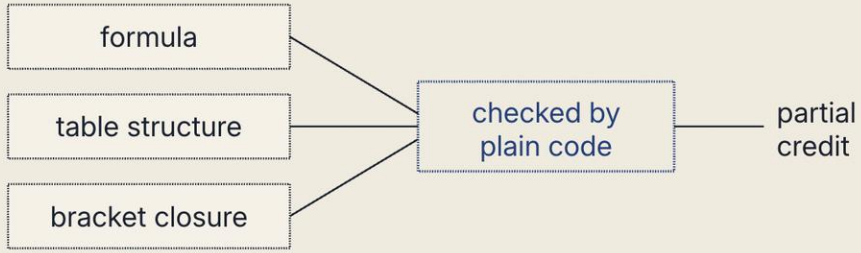
1 Guess ahead, then check FastMTP



The main model keeps the guesses it agrees with and fixes the first one it doesn't.

≈ **2×** faster on one L4 GPU, one request at a time. Same output as without it.

2 Scored by code RLVR



No judge model needed. Half-right still earns credit.

+7.4 points on olmOCR-Bench vs. DeepSeek-OCR, the model we started from (76.0 → 83.4).

arXiv:2609.03181 · L4 timing: vLLM, batch 1, eager mode (42.7 → 83.1 tok/s) · not a concurrent-serving number

Close to the top, with only 570M active parameters

OmniDocBench v1.6

higher

PaddleOCR-VL-1.6
scores higher than us

91.14

jina-ocr-v1
570M active · 3.4B total

89.78

Qwen3-VL-235B
235B general model

olmOCR-Bench

higher

chandra-ocr-2
scores higher than us

83.4

jina-ocr-v1
570M active · 3.4B total

76.0

DeepSeek-OCR
the model we started from

All numbers are from our arXiv paper, so you can check them.

arXiv:2609.03181 · default dynamic resolution · ladders show order, not scale · scores of higher-ranked systems are in the paper's tables

What it doesn't do

01 One page per call. For multi-page PDFs, the service splits and rejoins the pages.

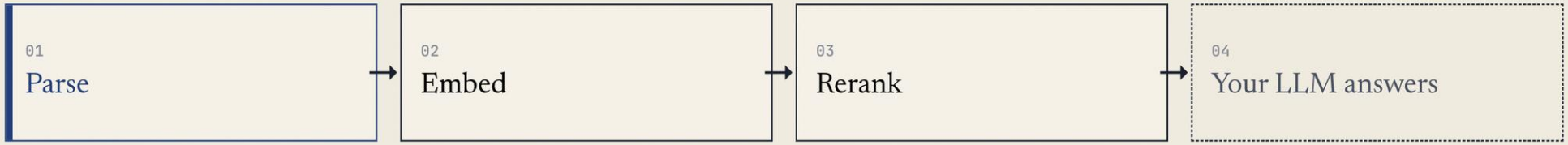
02 Output is Markdown. The demo page adds bounding boxes (third-party models).

03 The weights are non-commercial (CC BY-NC 4.0). For commercial use, call the API.

04 It needs a GPU. It's not built to run on CPUs.

Search and RAG start with turning pages into text

WHAT JINA BUILDS



Put the Markdown into **Elasticsearch** or pass it to **your agent**

```
curl https://r.jina.ai/<url> -H "Authorization: Bearer $JINA_API_KEY" \
  -H "X-Respond-With: jina-ocr-v1"
```

jina.ai/reader · huggingface.co/jinaai/jina-ocr-v1 · arxiv.org/abs/2609.03181

Try it on your hardest PDF this week.



Sa Zhang LinkedIn